Databricks Asset Bundles

DAB Complete Guide

Certificate Prep & Real-World Project Reference

v2.0 DAB | Databricks CLI | CI/CD Ready | IaC Approach

DJ (Dongjin Kim) · Data Engineer · 2026

Table of Contents

Chapter 1	DAB Overview & Core Concepts	3
Chapter 2	Project Structure & File Layout	5
Chapter 3	databricks.yml Deep Dive	7
Chapter 4	Resource Definitions (Job / Pipeline)	9
Chapter 5	CI/CD Architecture & Workflow	12
Chapter 6	Multi-Bundle & Domain Strategies	15
Chapter 7	bundle generate & CLI Commands	17
Chapter 8	Exam Cheat Sheet & Key Points	19

1 DAB Overview & Core Concepts

What is DAB?

Databricks Asset Bundles (DAB)는 Databricks의 공식 Infrastructure-as-Code(IaC) 프레임워크입니다. Job, DLT Pipeline, 노트북, ML 모델 등 Databricks 리소스를 YAML 파일로 정의하고 Git으로 버전 관리하여 dev/staging/prod 환경에 자동화된 배포를 가능하게 합니다.

> 시험 포인트 DAB는 Databricks 공식 권장 배포 방식으로, 자격증 시험에서도 핵심 주제로 다뤄집니다.

Core Principles

- Infrastructure-as-Code (IaC) — 모든 리소스를 코드(YAML)로 정의하고 Git으로 버전 관리
- Environment Isolation — dev / staging / prod 환경을 targets로 분리 관리
- Declarative Configuration — 상태를 선언하면 Databricks가 현재 상태와 diff를 계산해 적용
- CI/CD Integration — GitHub Actions, Azure DevOps 등과 네이티브 연동
- Collaboration — 팀 단위 개발, PR 리뷰, 승인 후 prod 배포 워크플로우 지원

Supported Resource Types

Resource Type	YAML Key	설명
Job / Workflow	jobs	Databricks Jobs (Task 기반 오케스트레이션)
DLT Pipeline	pipelines	Delta Live Tables / Lakeflow Pipelines
Notebook	notebooks	코드 에셋 (Python, SQL, Scala)
Cluster	clusters	재사용 가능한 클러스터 정의
ML Model	models	MLflow 모델 레지스트리 통합
Dashboard	dashboards	AI/BI Dashboard (신규)
Secret Scope	secrets	Secret 관리 (Vault 연동 등)

DAB vs. 기존 방식 비교

항목	기존 (UI/REST API)	DAB
버전 관리	불가 (UI 설정)	Git으로 완전 관리
환경 분리	수동 복사/관리	targets로 자동 분리

배포 자동화	수동 또는 스크립트	bundle deploy 단일 명령
팀 협업	어려움	PR 리뷰 기반 협업
Drift 감지	불가	선언적 상태 비교
롤백	수동	Git revert + redeploy

2 Project Structure & File Layout

권장 디렉토리 구조

```
my_project/ # Bundle root
├── databricks.yml # 메인 설정 파일 (필수)
├── resources/ # 리소스 정의 YAML
│ ├── jobs/
│ │ ├── ingestion_job.yml
│ │ └── transform_job.yml
│ └── pipelines/
│ ├── bronze_pipeline.yml
│ └── silver_pipeline.yml
├── src/ # 소스 코드
│ ├── notebooks/
│ │ ├── ingest.py
│ │ └── transform.sql
│ └── python/
│ └── utils.py
├── tests/ # 테스트 코드
│ ├── unit/
│ └── integration/
└── .github/
└── workflows/
└── deploy.yml # CI/CD 파이프라인
```

핵심 파일 역할

파일/디렉토리	역할	필수 여부
databricks.yml	번들 루트 설정, targets, workspace 정의	필수
resources/*.yml	Job, Pipeline 개별 리소스 정의	권장 (분리)
src/	노트북, Python 코드, SQL 파일	권장
tests/	Unit/Integration 테스트	권장
.github/workflows/	GitHub Actions CI/CD 파이프라인	CI/CD 필요시

Bundle Init — 빠른 시작

```
# 1. Databricks CLI 설치
pip install databricks-cli
```

```
# 2. 인증 설정
databricks configure --token

# 3. 새 번들 프로젝트 초기화
databricks bundle init

# 4. 템플릿 선택 (interactive)
# ❯ default-python / default-sql / dbt-sql

# 5. 검증
databricks bundle validate

# 6. 개발 환경 배포
databricks bundle deploy --target dev
```

3 databricks.yml Deep Dive

전체 구조 (Annotated)

```yaml
# databricks.yml
bundle:
  name: my_data_project # 번들 고유 이름

variables:
  env:
    default: dev
  catalog:
    default: dev_catalog

workspace:
  host: https://<workspace>.azuredatabricks.net

resources:
  jobs:
    ingestion_job:
      source: resources/jobs/ingestion_job.yml
  pipelines:
    bronze_pipeline:
      source: resources/pipelines/bronze_pipeline.yml

targets:
  dev:
    mode: development # 핵심! dev prefix 자동 부여
    default: true
    variables:
      catalog: dev_catalog
      env: dev

  staging:
    mode: production
    workspace:
      host: https://<staging>.azuredatabricks.net
    variables:
      catalog: staging_catalog

  prod:
    mode: production
    workspace:
      host: https://<prod>.azuredatabricks.net
    variables:
```

```
      catalog: prod_catalog
    run_as:
      service_principal_name: sp-databricks-prod
```

mode: development vs mode: production

속성	mode: development	mode: production
리소스 이름	[dev 유저명] prefix 자동 추가	정의된 이름 그대로 사용
Schedule	자동 pause (실수 방지)	정의대로 활성화
파일 경로	/dev// 하위에 격리	설정된 경로 사용
용도	개인 개발/테스트 환경	Staging/Prod 배포
동시 사용	여러 개발자 독립적으로 사용 가능	단일 공유 환경

시험 핵심 mode: development는 시험 단골 문제! 리소스 이름에 [dev username] prefix가 붙어 prod 리소스와 완전히 격리됩니다. Schedule도 자동으로 pause됩니다.

변수 치환 문법

```
resources:
  jobs:
    my_job:
      name: "${bundle.name}-job-${var.env}"
      tasks:
        - task_key: main
          notebook_task:
            notebook_path: "${workspace.file_path}/src/notebooks/main"
            base_parameters:
              catalog: "${var.catalog}"
              environment: "${var.env}"

# 런타임 오버라이드
databricks bundle deploy --target prod --var="catalog=prod_v2"
```

4 Resource Definitions (Job / Pipeline)

Job 정의 예시

```yaml
# resources/jobs/ingestion_job.yml
resources:
  jobs:
    ingestion_job:
      name: "ingestion-job-${var.env}"

      job_clusters:
        - job_cluster_key: main_cluster
          new_cluster:
            spark_version: "15.4.x-scala2.12"
            node_type_id: "i3.xlarge"
            num_workers: 2

      tasks:
        - task_key: ingest_orders
          job_cluster_key: main_cluster
          notebook_task:
            notebook_path: "${workspace.file_path}/src/notebooks/ingest_orders"
            base_parameters:
              catalog: "${var.catalog}"

        - task_key: ingest_products
          depends_on:
            - task_key: ingest_orders
          job_cluster_key: main_cluster
          notebook_task:
            notebook_path: "${workspace.file_path}/src/notebooks/ingest_products"

      schedule:
        quartz_cron_expression: "0 0 2 * * ?"
        timezone_id: "America/New_York"
        pause_status: UNPAUSED

      email_notifications:
        on_failure:
          - data-engineering@company.com
```

DLT Pipeline 정의 예시

```yaml
# resources/pipelines/bronze_pipeline.yml
resources:
  pipelines:
    bronze_pipeline:
      name: "bronze-ingestion-pipeline-${var.env}"
      target: "${var.catalog}.bronze"

      configuration:
        pipelines.numUpdateRetryAttempts: "3"

      libraries:
        - notebook:
            path: "${workspace.file_path}/src/notebooks/bronze_orders"
        - notebook:
            path: "${workspace.file_path}/src/notebooks/bronze_products"

      serverless: true # 클러스터 관리 불필요 (권장)
      channel: "CURRENT"
      continuous: false # true=Streaming, false=Triggered

      notifications:
        - email_recipients:
            - de-team@company.com
          alerts:
            - on-update-failure
```

Job vs DLT Pipeline 선택 기준

기준	Databricks Job	DLT Pipeline
주요 용도	일반 오케스트레이션, 배치 처리	데이터 변환, Medallion 구축
데이터 품질	수동 구현 필요	Expectations 내장
의존성 관리	Task 간 DAG 명시	자동 lineage 추적
스트리밍	Spark Structured Streaming 직접	STREAMING TABLE 선언
권장 패턴	ML, 복잡한 워크플로우	ETL, Medallion Architecture

5 CI/CD Architecture & Workflow

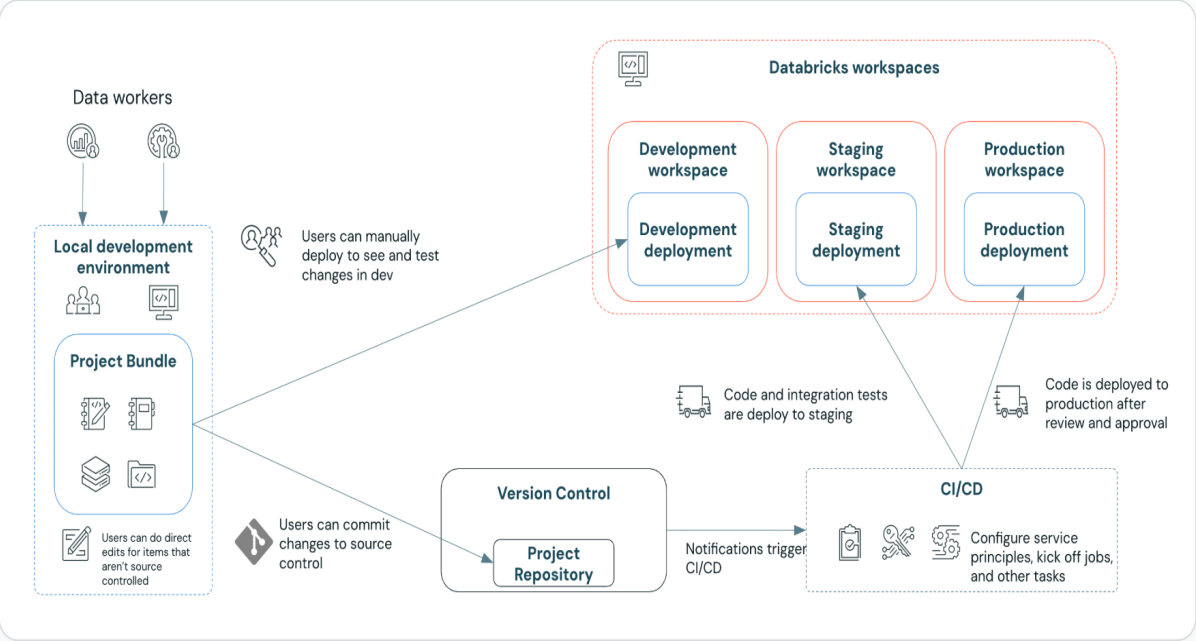


Figure 1. DAB CI/CD Architecture — Local Dev → Version Control → CI/CD → Staging/Production

전체 CI/CD 플로우

단계	트리거	액션	환경
1	로컬 개발	bundle deploy --target dev (수동)	Dev Workspace
2	PR 생성	bundle validate + unit tests (자동)	CI Runner
3	develop merge	bundle deploy --target staging (자동)	Staging Workspace
4	Integration Test	bundle run --target staging (자동)	Staging Workspace
5	main branch merge	승인 후 bundle deploy --target prod	Prod Workspace

GitHub Actions 완전한 예시

```yaml
# .github/workflows/deploy.yml
name: DAB CI/CD Pipeline

on:
  push:
    branches: [develop, main]
```

```yaml
  pull_request:
    branches: [develop, main]

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install Databricks CLI
        run: pip install databricks-cli
      - name: Validate Bundle
        run: databricks bundle validate
        env:
          DATABRICKS_HOST: ${{ secrets.DATABRICKS_HOST_DEV }}
          DATABRICKS_TOKEN: ${{ secrets.DATABRICKS_TOKEN_DEV }}

  deploy-staging:
    needs: validate
    if: github.ref == 'refs/heads/develop'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Deploy to Staging
        run: databricks bundle deploy --target staging
        env:
          DATABRICKS_HOST: ${{ secrets.DATABRICKS_HOST_STAGING }}
          DATABRICKS_TOKEN: ${{ secrets.DATABRICKS_TOKEN_STAGING }}
      - name: Run Integration Tests
        run: databricks bundle run integration_test_job --target staging
        env:
          DATABRICKS_HOST: ${{ secrets.DATABRICKS_HOST_STAGING }}
          DATABRICKS_TOKEN: ${{ secrets.DATABRICKS_TOKEN_STAGING }}

  deploy-prod:
    needs: deploy-staging
    if: github.ref == 'refs/heads/main'
    environment: production # GitHub Environment 승인 게이트
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Deploy to Production
        run: databricks bundle deploy --target prod
        env:
          DATABRICKS_HOST: ${{ secrets.DATABRICKS_HOST_PROD }}
          DATABRICKS_TOKEN: ${{ secrets.DATABRICKS_TOKEN_PROD }}
```

6 Multi-Bundle & Domain Strategies

번들 구성 전략 비교

전략	구조	장점	단점	권장 상황
모노레포	1 bundle, N resources	단순, 일괄 배포	팀 간 충돌 가능	소규모 팀, 강한 의존성
도메인 분리	N bundles by domain	독립 배포, 자율성	중복 설정 가능	MSA, 도메인팀 분리
레이어 분리	Ingestion / Transform bundles	레이어별 배포 주기	레이어 간 조율 필요	Medallion Architecture

도메인 분리 예시 (권장)

```
data_platform/
├── orders_domain/
│   ├── databricks.yml
│   └── resources/
│   ├── orders_dlt_pipeline.yml # DLT: orders bronze
│   └── orders_transform_job.yml # Job: orders silver/gold
│
├── products_domain/
│   ├── databricks.yml
│   └── resources/
│   ├── products_dlt_pipeline.yml
│   └── products_transform_job.yml
│
└── customers_domain/
    ├── databricks.yml
    └── resources/
        ├── customers_dlt_pipeline.yml
        └── customers_transform_job.yml

# 도메인 독립 배포
cd orders_domain && databricks bundle deploy --target prod
cd products_domain && databricks bundle deploy --target prod
```

Medallion Architecture — 레이어 분리

```
# ingestion_bundle/databricks.yml (DLT 3개)
bundle:
  name: ingestion-layer
```

```
resources:
  pipelines:
    bronze_orders:
      source: resources/bronze_orders_pipeline.yml
    bronze_products:
      source: resources/bronze_products_pipeline.yml
    bronze_customers:
      source: resources/bronze_customers_pipeline.yml

# transform_bundle/databricks.yml (Job 3개)
bundle:
  name: transform-layer
resources:
  jobs:
    orders_transform:
      source: resources/orders_transform_job.yml # Silver/Gold
    products_transform:
      source: resources/products_transform_job.yml
    customers_transform:
      source: resources/customers_transform_job.yml
```

7 bundle generate & CLI Commands

bundle generate — 기존 리소스 추출

Databricks UI에서 직접 YAML export 기능은 없습니다. bundle generate 명령을 사용하면 기존 Job/Pipeline을 DAB YAML 형식으로 자동 변환할 수 있습니다.

```
# 기존 Job -> YAML 변환
databricks bundle generate job \
  --existing-job-id 123456789 \
  --config-dir ./resources/jobs

# 기존 DLT Pipeline -> YAML 변환
databricks bundle generate pipeline \
  --existing-pipeline-id abc-def-123 \
  --config-dir ./resources/pipelines

# JSON으로 먼저 확인
databricks jobs get --job-id 123456789 > job.json
databricks pipelines get --pipeline-id abc-def-123 > pipeline.json
```

마이그레이션 단계 (기존 환경 -> DAB)

단계	작업	명령어
1	기존 Job/Pipeline ID 확인	databricks jobs list
2	번들 프로젝트 초기화	databricks bundle init
3	YAML 자동 생성	bundle generate job --existing-job-id
4	생성된 YAML 검토/수정	수동 편집
5	유효성 검사	databricks bundle validate
6	Dev 배포 확인	bundle deploy --target dev
7	Git 커밋 후 CI/CD 연동	git push

전체 CLI 명령어 레퍼런스

명령어	설명
databricks bundle init	새 번들 프로젝트 생성 (템플릿 선택)

databricks bundle validate	YAML 문법 및 참조 유효성 검사
databricks bundle deploy	현재 번들을 workspace에 배포
databricks bundle deploy --target prod	특정 target에 배포
databricks bundle deploy --force-lock	배포 lock 강제 해제 후 배포
databricks bundle run	배포된 Job/Pipeline 실행
databricks bundle destroy	배포된 모든 리소스 삭제
databricks bundle generate job	기존 Job -> YAML 변환
databricks bundle generate pipeline	기존 Pipeline -> YAML 변환
databricks bundle summary	번들 리소스 요약 출력

8 Exam Cheat Sheet & Key Points

시험 핵심 포인트 총정리

mode: development	리소스 이름에 [dev username] prefix 자동 추가, Schedule 자동 pause, 파일 경로 /dev// 하위 격리 -> prod와 완전히 분리됨
mode: production	리소스 이름 그대로 사용, Schedule 정의대로 활성화. staging에도 production mode 사용
bundle generate	기존 UI Job/Pipeline을 DAB YAML로 자동 변환. UI에서 직접 yml export 기능 없음
bundle validate	YAML 문법 오류, 변수 참조 오류 등을 배포 전에 검사. CI/CD 첫 단계로 필수
bundle destroy	배포된 모든 리소스 삭제. 되돌리기 어려우므로 주의. dev 환경 정리 시 사용
Service Principal	Prod 배포는 개인 토큰 대신 SP(Service Principal) 사용 권장. run_as 설정으로 지정
databricks.yml 위치	Bundle root에 위치해야 함. 이 파일이 있는 디렉토리가 번들 루트로 인식됨
변수 치환 문법	${var.name}, ${bundle.name}, ${workspace.file_path} 등 사용. --var 옵션으로 런타임 오버라이드
targets 개념	dev/staging/prod 환경을 별도 workspace로 완전히 분리. 각 target별 독립 배포 가능
serverless pipeline	DLT Pipeline에서 serverless: true 설정 시 클러스터 관리 불필요. Databricks 권장 옵션

Quick Reference Card

```
# 인증
export DATABRICKS_HOST=https://<workspace>.azuredatabricks.net
export DATABRICKS_TOKEN=<token>

# 개발 사이클
databricks bundle validate # 검사
databricks bundle deploy --target dev # 개발 배포
databricks bundle run my_job # 실행
databricks bundle destroy --target dev # 정리

# 기존 리소스 마이그레이션
databricks bundle generate job --existing-job-id <ID>
databricks bundle generate pipeline --existing-pipeline-id <ID>
```

```
# CI/CD
# develop branch push -> staging 자동 배포
# main branch push -> prod 수동 승인 후 배포
```

시험 최종 정리 자격증 시험에서 DAB 관련 문제는 mode 설정, bundle 명령어, targets 개념, Service Principal 사용이 자주 출제됩니다. 특히 mode: development의 동작 방식을 정확히 이해하세요.